

LANGCHAIN + STREAMLIT PROJECT

Building a ChatGPT Clone

An end-to-end guide to creating an interactive conversational AI application with persistent memory and dynamic LLM orchestration.

Architecture & Core Concepts

Combining Streamlit's reactive UI engine with LangChain Expression Language (LCEL) and stateful message stores.

Core System Architecture

1. Streamlit Frontend

Handles user interactions, API key validation, real-time message rendering (`st.chat_message`), and session state management.

Maintains runtime authentication status and chat history arrays seamlessly without page reloads.

2. LangChain Pipeline

Orchestrates the prompt engineering, OpenAI model connection (`ChatOpenAI`), output parsing (`StrOutputParser`), and memory history.

Constructs a declarative LCEL chain wrapped in `RunnableWithMessageHistory` for session isolation.

Key Application Features



API Gatekeeper

Validates OpenAI keys with a test invocation prior to session setup, shielding backend calls from unauthorized execution.



Session Memory

Utilizes ``InMemoryChatMessageHistory`` to remember prior interactions across multiple conversational turns.



Dynamic Controls

Enables on-the-fly model switching (GPT-3.5/GPT-4o), temperature adjustments, and customized system prompts.

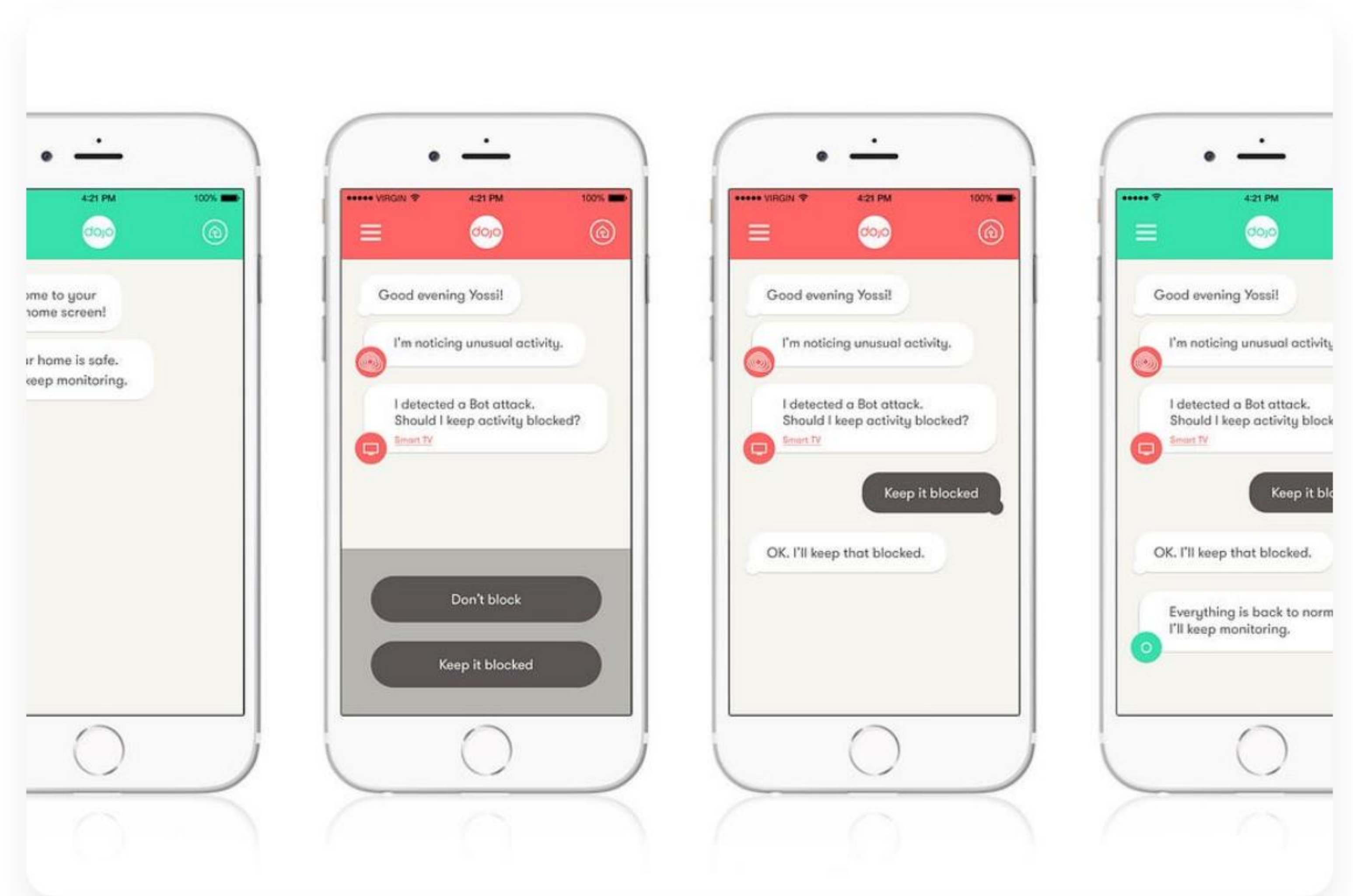
Stateful Conversational Memory

`RunnableWithMessageHistory` Integration

Memory is kept separate from standard prompt inputs, allowing clean template compilation.

Workflow:

- Stores messages by unique `session\_id`.
- Injects `chat\_history` into the `MessagesPlaceholder`.
- Appends user inputs and AI completions automatically.



CLI vs. Streamlit Implementation

Aspect	Jupyter / CLI Script	Streamlit Web Application
User Interface	Terminal prompt (<code>input()</code>)	Web GUI (<code>st.chat_input</code> & chat bubbles)
API Key Setup	Static <code>.env</code> file via <code>dotenv</code>	Interactive front gate validation form
State Management	Local Python memory loop	<code>st.session_state</code> + Resource caching
User Controls	Hardcoded parameters	Interactive sidebar sliders & select boxes

Configurable Controls

4+

OpenAI Models Supported

Granular Output Control

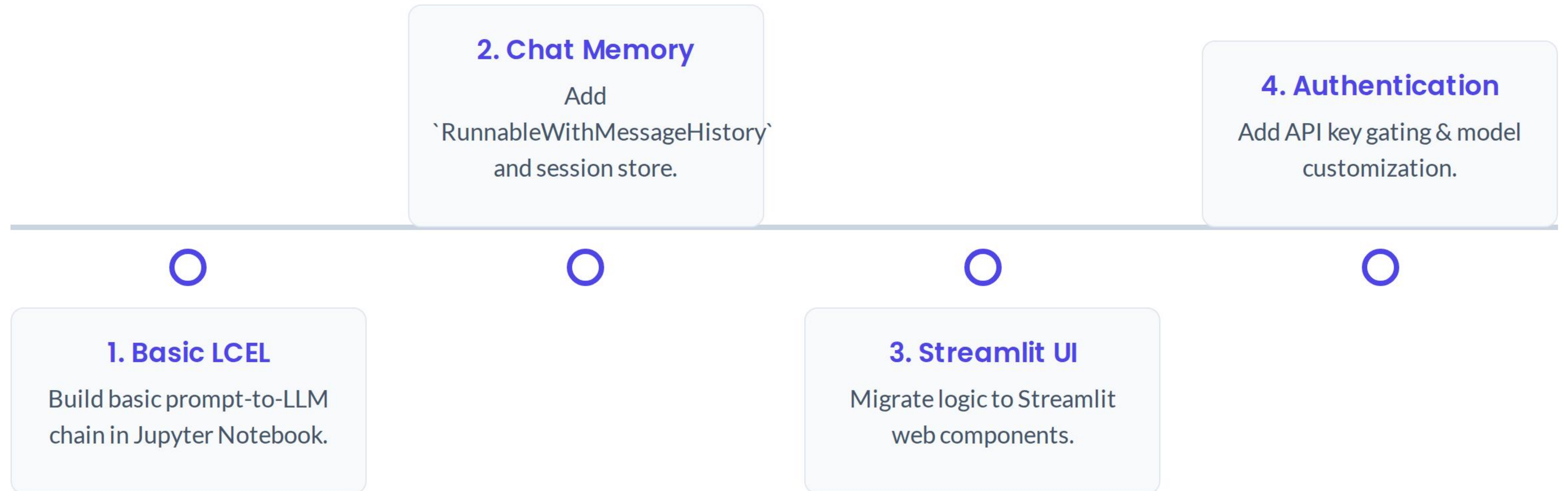
The app supports real-time switching across key models: **gpt-3.5-turbo**, **gpt-4o**, **gpt-4o-mini**, and **gpt-4-turbo**.

Temperature can be adjusted continuously between 0 . 0 (strictly factual) and 2 . 0 (highly creative).

Security & Optimization Practices

-  **Ephemeral Key Storage:** Keys are kept only in memory (`st.session_state`) during the active session and never stored on disk.
-  **Resource Caching:** `@st.cache_resource` prevents redundant chain rebuilds on every Streamlit re-run.
-  **Interactive Session Clearing:** Includes a dedicated "Clear Chat" control that flushes session history and resets the state cleanly.
-  **Error Handling:** Catches API exceptions gracefully to inform users without crashing the application interface.

Development Roadmap



Questions & Discussion

Thank you for exploring the LangChain & Streamlit ChatGPT Clone project!

Built with [LangChain](#) | [OpenAI](#) | [Streamlit](#)

Image Sources



https://miro.medium.com/v2/resize:fit:1200/1*uzNyRenza_4A-dqv2XrTiw.jpeg

Source: uxdesign.cc